



'22 딥러닝 알고리즘 기반 욕설 감지 필터

박미현

백지현

송현승

이창용

제갈덕

김주영

01. 프로젝트 개요

1-1. 프로젝트 기획 배경 및 목표

프로젝트 기획 배경

게임 내 욕설은 온라인 게임의 시작부터 역사를 같이 해왔다. 최근에는 스마트폰 등 전자기기의 보편화로 인터넷의 접근성이 좋아져 어린이나 청소년이 온라인상에서의 욕설이나 비방에 노출되기 쉬워졌다. 실제로 한국콘텐츠진흥원의 '2021 게임 이용자 실태조사 보고서'에 따르면 나이가 어릴수록 채팅으로 욕설 피해를 당한 비율이 높았다.

뿐만 아니라 악플 또한 근절되어야 할 인터넷 문화 중 하나다. 사이버 명예훼손죄와 모욕죄 건수는 2014년 8,880건에서 2019년 16,633건으로 87% 증가했고, 김찬호 성공회대 교수의 연구에 따르면 한국의 인터넷 게시판 댓글에서 악플 대 선플의 비율은 4:1로 인신공격과 욕설에 대한 규제가 필요한 수준이었다. 실제로 이번 프로젝트에 사용한 데이터를 형태소 단위로 토큰화하여 시각화한 결과, 비속어가 큰 비율을 차지했다.



모델 학습에 사용한 데이터 워드클라우드

현재 게임 업체, 포털 측에서도 욕설과 비방을 잡아내기 위한 ai를 이용하고 있다. 네이버의 'ai 클린봇'은 기사 댓글에 부적절한 댓글이 달리면 자동으로 숨김 처리 하고, 왓챠도 콘텐츠 감상평에 AI 모니터링 기술을 적용해 부적절한 코멘트의 경우 블라인드 처리한다. 선거관리 위원회는 2022년 대선부터 ai를 이용해 욕설과 비하를 잡아내는 지능형 사이버 선거범죄 대응 시스템을 운용한다.

그러나 사실 위의 경우처럼 ai 욕설 필터가 실제로 적용되는 사례는 드물다. 오히려 개인 프로젝트나 논문이 실사례에 비해 비교적 많다. 욕설은 변형이 쉬워 사용자들이 새로운 형태의 욕설을 만들어내기 때문이다. 새 욕설 데이터가 생기면 새로 라벨링을 해야 하기 때문에 번거롭고 인적 자원이 계속 소모된다. 또한 현 시점에서 존재하는 욕설을 잡아내는 ai를 개발해도, 사용자가 해당 ai의 패턴을 파악해 필터링을 뚫는 욕설을 만들면 그 이후까지 잡아내는 모델을 만드는 것은 불가능에 가깝다. 이러한 이유로 지금까지 실제 도입된 사례가 적으며, ai 관련 회사들이 계속해서 모델을 개발 중이다. 블루오션인 동시에 개척해야 하는 황무지인 것이다.

흔히 게임 업체나 커뮤니티 등에서 사용하는 금칙어 리스트 설정 방식으로는 변형된 욕설이나 비방을 잡아내기 힘들다. 알고리즘 자체는 간단하지만, 욕이 아닌 단어에 금칙어가 포함된 경우 이용에 불편을 겪게 된다. 구독형 프라이빗 메세지 서비스 '버블'이나 '유니버스'의 경우 금칙어 리스트 설정 방식을 사용 중인데, 욕설 및 비방과 관련없는 단어가 제재되고 정상 변형된 욕은 잡지 못하는 상황이 자주 발생한다.

← JYP Bubble banned words

009sex
100boG
100bozy
1090tv
10알
10알년
10자속
10팔새까
10할
10sextv
18님
18노마
18농
18누마
18새까
18세가
18세까
18쌌끼
18sex

'버블'에서 사용중인 금칙어 리스트

마라~~상~~귀 밀크~~쉐~~이크 많이 먹고 건강해~~야~~해



단어에 금칙어가 포함되어 이용이 제재되는 경우

색기

04:50

욕설, 비방은 처벌될 수 있습니다



변형된 욕을 잡지 못하는 경우

따라서 이 프로젝트에서는 딥러닝을 이용해 금칙어 리스트를 지정해주지 않아도 욕을 잡아내는 욕설 필터를 만들고자 한다. 같은 단어도 문맥에 따라 욕설 여부를 판단하게 하고, 특수문자를 집어넣거나 자모음을 변경해 만든 새로운 욕설을 욕설로 판단하도록 한다. 이는 인적 자원의 소모를 감소시키고 욕설 제재 프로세스가 더욱 경제적으로 운영되게 함으로써, 게임 유저의 만족도와 시스템의 효율성을 높일 수 있을 것이다.

프로젝트 목표

- 금치어 리스트를 사용하지 않고, 딥러닝을 이용해 욕설 감지 필터를 만든다.
- '^' | 발', '미친' 등 특수 문자를 이용해 변형한 욕을 욕설로 판별한다.
- '전성기' 등 비속어로 판별할 만한 문구가 포함되었지만 욕이 아닌 단어를 욕설로 판별하지 않도록 한다.

1-2. 구성원 및 역할

- 제갈덕 : 팀장, 데이터 라벨링, 데이터 전처리, 장고 서비스 구현, 발표
- 김주영 : 데이터 라벨링, 장고 서비스 구현
- 박미현 : 데이터 라벨링, 데이터 전처리, 모델링, 보팅
- 백지현 : 크롤링 및 db 구현, 데이터 라벨링, 장고 서비스 구현
- 송현승 : 데이터 라벨링, 데이터 전처리, 모델링, ppt
- 이창용 : 크롤링 및 db 구현, 데이터 라벨링, 모델링, 보팅

02. 프로세싱

2-1. 데이터 명세

데이터 출처, 형태(csv, json, xml 등), 크기

2-2. 데이터 수집/분석 시스템 구축

커뮤니티 게시물 데이터 크롤링

게임 채팅 데이터 수집이 현실적으로 어려워, 게임과 관련이 높은 커뮤니티를 선정하여 40,000개의 글 제목을 크롤링했다. 욕설 데이터와 비욕설 데이터의 양적 균형을 맞추기 위해 추후 욕설 데이터를 2차로 수집하였다.

- 디씨인사이드 30,000개
- 인벤 15,000개
- 에펨코리아 게시판 15,000개
- 네이트판 15,000개

데이터 라벨링

1인당 약 15,000개의 데이터에 라벨링을 진행했다. 욕설이면 1, 비욕설이면 0으로 라벨링했다. 욕설 판단 기준은

<https://www.notion.so/c1ecb7cc52d446cc93d928d172ef8442> 을 참고했다. 크게 2가지 기준을 정하였으며, 두 기준 중 하나라도 해당되는 경우 욕설로 판단했다. 기준은 다음과 같다.

1. 특정 계층(성별, 나이, 정치, 종교, 장애 등)에 대한 편견/차별적인 발언,
2. 특정 대상에 대한 모욕/공격적인 발언

크롤링 과정 상 문제발생과 해결방법

1. 크롤링을 위해 사용한 Google Colabortory의 IP가 커뮤니티 웹 서버에서 밴을 당하는 문제 발생
 - 원인(추정) : 크롤링 누적 시간이 길어지고 여러 페이지를 크롤링하면서 일정 주기로 페이지를 이동하는 작업이 봇으로 인식되어짐.
 - 해결과정 : 처음 시도했던 방법은 sleep 값을 10으로 고정하여 크롤링을 시도하였다. 인벤 게시판과 네이트판의 경우 이 방법으로도 다량의 데이터를 크롤링하는데 차질없이 진행이 되었다. 이후 동일한 방법으로 에펠타의 게시판을 크롤링하고자 하였으나, 10초로 설정하였을 때, 5000건을 총 250회로 분할하여 크롤링하는 도중 약 40회째에 에펠타 웹 서버로부터 IP밴을 당하였다. (Colab 상에서의 IndexError: list index out of range 로 나타난다.) 이에 대응하기 위해 time.sleep 값을 20, 30, 60 등으로 더욱 길게 부여해보았으나, 여전히 봇으로 간주되어 밴 당하는 문제가 지속
 - 해결방법 : time.sleep값을 최소 60으로 하고, 새로 생성한 여러 난수의 곱으로 처리하여 IP밴을 회피하였다.
2. 일부 크롤링 데이터에 포함된 공백만이 있는 데이터와, 정상 데이터에 포함된 수많은 \n 이 크롤링 과정에서 제거되지 않는 문제
 - 원인(추정): 크롤링할 때 해당 게시물 데이터의 .text 부분만을 가져오도록 하였으나 해당 부분까지 텍스트의 일부로 인식됨
 - 해결과정: 수작업으로 제거하려고 하였으나, 해당 되는 케이스가 많은 것을 확인하고 pandas의 기능을 이용해 공백을 제거할 방법을 찾음
 - 해결방법: str.strip().replace("\n","")를 하여 앞 뒤 공간과 줄바꿈을 제거하고, replace(", np.nan)으로 값이 존재하지 않는 데이터 행을 삭제하였다.
3. 크롤링 한 로우데이터를 csv 파일로 저 인코딩 과정에서 정상적으로 저장되지 않는(한글 깨짐 현상) 문제 발생

- 원인 : encoding 방식을 utf-8으로 설정
 - 해결과정 : 동일하게 csv 깨짐 문제가 발생한 팀원에게 해결 방법을 공유 받음.
 - 해결방법: encoding 방식을 utf-8 → utf-8-sig 로 변경하였더니 csv 파일에 한글이 정상 저장하였음
4. 일부 커뮤니티의 경우 단순 request 나 beautifulsoup만으로는 크롤링이 정상적으로 되지 않는(공백 저장) 문제 발생
- 원인: 서버에서 크롤링을 염두에 두고 사전에 봇을 제한하여 둔 것으로 추정
 - 해결과정: 해당 웹사이트의 Html 상세 정보에서 봇의 활동을 제한하여 두었음을 확인함
 - 해결방법: headers 에 'User-Agent' 를 입력하여, 봇이 아닌 사람으로 인식되어 크롤링을 정상적으로 할 수 있게 됨

2-3. 데이터 분석 및 시각화

데이터 전처리

1. 데이터 정제

숫자나 특수문자를 이용해 변형한 욕설을 감지하기 위해 특수문자를 제거하지 않는다. 그러나 데이터에 이모지가 들어가면 임베딩 과정에 영향을 주거나 모델 성능을 저하시킬 수 있으므로 제거했다. 이모지 제거에는 파이썬 emoji 라이브러리를 사용했다.

디씨인사이드에서 수집한 데이터의 경우 글 제목과 함께 댓글 개수까지 수집되었다. 이 또한 모델 학습에 영향을 줄 수 있으므로 제거했다. 댓글 수 제거는 파이썬 re 라이브러리와 정규표현식을 사용했다.

데이터 정제는 아래와 같은 순서로 진행되었고, 총 데이터의 개수는 58,312개에서 정제 후 56,816개가 되었다.

	records
0	외모에 따른 한녀 스킨트리 짜봤다. ?->?->?or?♂+ [78]
1	👉 방금 지하철 도태한녀 목격설.txt [71]
2	그브♥트페 아니시발 오피셜이었음??
3	♥+같은강사 더 저렴하게 이용하세요 ㅇㅈㄴㄴ♥+

수집한 데이터

- a. NaN값 삭제
- b. 문자열 중 이모티콘과 댓글 개수 표시([n]) 제거

```
!pip install emoji
import emoji

def extract_emojis(text):
    for c in text:
        if c in emoji.UNICODE_EMOJI['en']:
            text = re.sub(c, "", text)

    return text

data['records'] = data['records'].map(extract_emojis)
# 이모티콘 지우기
data['records'] = data['records'].str.replace(r'\\[\d+\\]', "") # 댓글개수 지우기
data['records'] = data['records'].str.replace("\\xa0", "",) # 띄어쓰기 특수문자 지우기
```

이모티콘, 댓글 개수 제거 코드

- c. 공백 및 중복 데이터제거

```
data.dropna(inplace=True) # nan값 삭제
data.drop_duplicates(subset=['records'], inplace=True) # 중복값 삭제
```

공백, 중복 데이터 제거 코드

records

```
0 외모에 따른 한녀 스킨트리 짜봤다. ?->?->?or?+
1          방금 지하철 도태한녀 목격썰.txt
2          그브트페 아니시발 오피셜이었음??
3 +같은강사 더 저렴하게 이용하세요 ㄹㄱ ㄷㄹ+
```

데이터 정제 결과

2. 토큰화

모델링을 진행하기 전, 문장 토큰화를 진행한다. 어절, 형태소, 음절, 자모 총 4가지 단위로 토큰화해보았고, 모델 프로토타입 단계에서 어절과 음절을 제외해 최종적으로 형태소, 자모 2가지를 사용하기로 했다.

a. 어절 단위 토큰화

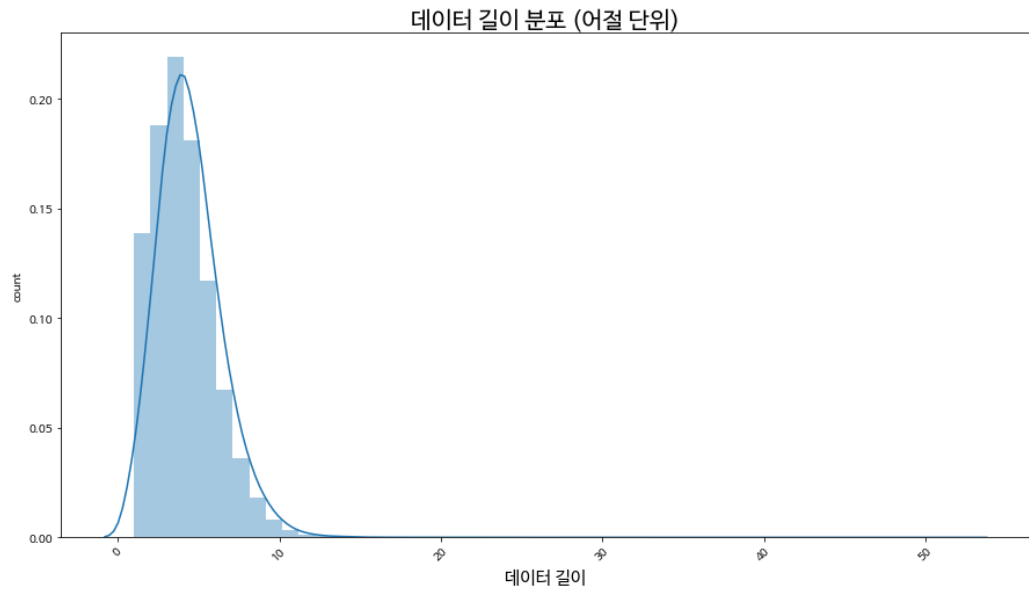
i. tensorflow의 Tokenizer을 사용하여 어절 단위 토큰화를 진행했다.

```
from tensorflow.keras.preprocessing.text import Tokenizer

token = Tokenizer()
token.fit_on_texts(X_train)
```

어절단위 토큰화 코드

ii. 데이터 길이의 분포를 히스토그램으로 시각화한 결과, 어절이 10개 이하인 데이터가 대부분이었다.



데이터 길이 분포 히스토그램

- iii. 어절 사용 빈도를 워드클라우드로 나타냈다. 띄어쓰기 단위로 단어를 구분했기 때문에, '새끼'와 '이새끼'의 경우 동일한 의미의 명사가 쓰였음에도 다른 단어로 취급된다.



어절 단위 워드클라우드

iv. 정수 인코딩

- 단어 집합(vocabulary)의 크기 : 77,978

- 등장 빈도가 1번 이하인 희귀 단어의 수: 61,775
- 단어 집합에서 희귀 단어의 비율: 79.22%
- 전체 등장 빈도에서 희귀 단어 등장 빈도 비율: 34.78%

희귀단어의 등장 빈도와 단어 집합에서 차지하는 비율이 꽤 높아 희귀단어를 제거하지 않고 인코딩을 진행했다.

```
word_X_train = token.texts_to_sequences(X_train)
word_X_test = token.texts_to_sequences(X_test)
```

정수 인코딩 코드

v. 패딩

전체 샘플 중 어절이 10개 이하인 샘플의 비율이 99.35%로 데이터의 대부분을 차지했다. 글자가 100개 이상인 특이 데이터를 살펴본 결과, 노래가사나 욕설이 포함되지 않아 label=0인 데이터 뿐이었다. 따라서 패딩 길이를 10으로 설정했다.

```
max_len = 10
word_X_train = pad_sequences(word_X_train, maxlen = max_len) # 시리즈인지 데이터프레임인지 리스트인지 타입이 중요. 패드 시퀀스에 뭐 들어가는지 확인 필요
word_X_test = pad_sequences(word_X_test, maxlen = max_len)
```

패딩 코드

vi. 프로토타입 모델

```

model = Sequential()
model.add(Embedding(total_cnt, 100))
model.add(LSTM(128))
model.add(Dense(1, activation='sigmoid'))

es = EarlyStopping(patience=4, restore_best_weights=True)
mc_w = ModelCheckpoint('./model/best_model_mean_words_word.h5', save_best_only=True)

model.compile(optimizer='rmsprop', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(word_X_train, y_train, epochs=100, callbacks=[es, mc_w], batch_size=64, validation_split=0.2)

print("\n Accuracy: %.4f" % (model.evaluate(word_X_test, y_test)[1]))
# Accuracy: 0.8410

```

어절 단위 토큰화, lstm 모델 코드

어절 단위로 토큰화하여 lstm 신경망을 이용한 모델은 정확도 84.1%를 가지며 좋지 않은 성능을 보였다. 따라서 추후 모델링에 어절단위 토큰화는 사용하지 않기로 결정했다.

b. 형태소 단위 토큰화

- i. KoNLPy 라이브러리의 Okt 형태소 분석기를 사용하여 문장을 형태소 단위로 토큰화했다.

```

from konlpy.tag import Okt
from tqdm import tqdm

okt = Okt()

train_record = []
for sentence in tqdm(X_train):
    tokenized_sentence = okt.morphs(sentence, stem

```

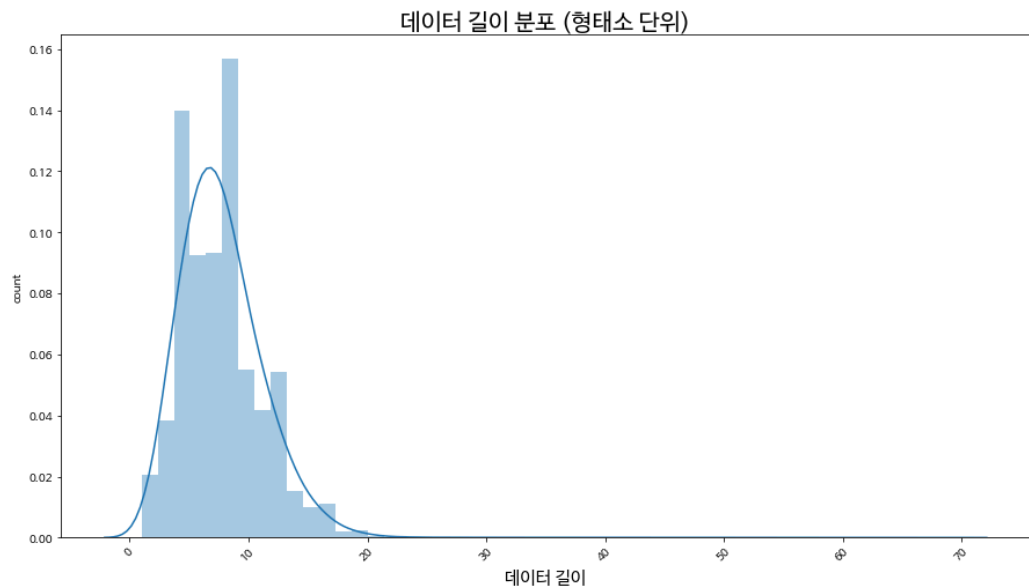
```

=True) # 토큰화
    train_record.append(tokenized_sentence)

test_record = []
for sentence in tqdm(X_test):
    tokenized_sentence = okt.morphs(sentence, stem
=True) # 토큰화
    test_record.append(tokenized_sentence)

```

- ii. 데이터 길이의 분포를 히스토그램으로 시각화한 결과, 형태소가 20개 이하인 데이터가 전체의 99.8%였다.



- iii. 형태소 사용 빈도를 워드클라우드에 나타냈다. '새끼'의 경우 어절 단위 워드클라우드에서는 '새끼'와 '이새끼'가 다른 단어로 취급되어 사용 빈도 수가 적었지만, 형태소 워드클라우드에서는 같은 단어로 취급되어 제일 높은 빈도수를 가진다.



iv. 정수 인코딩

- 단어 집합(vocabulary)의 크기 : 24,626
- 등장 빈도가 1번 이하인 희귀 단어의 수: 12,216
- 전체 등장 빈도에서 희귀 단어 등장 빈도 비율: 3.94%
- 희귀단어의 등장 빈도와 단어 집합에서 차지하는 비율이 꽤 높아 희귀단어를 제거하지 않고 인코딩을 진행했다.

```
morpheme_X_train = tokenizer.texts_to_sequences(train_record) #텍스트 시퀀스를 정수 시퀀스로 변환
morpheme_X_test = tokenizer.texts_to_sequences(test_record)
```

v. 패딩

전체 샘플 중 어절이 20개 이하인 샘플의 비율이 99.8%로 데이터의 대부분을 차지했다. 따라서 패딩 길이를 20으로 설정했다.

```
morpheme_X_train = pad_sequences(morpheme_X_train,
maxlen = max_len) # 시리즈인지 데이터프레임인지 리스트인
지 타입이 중요. 패드 시퀀스에 뭐 들어가는지 확인 필요
morpheme_X_test = pad_sequences(morpheme_X_test, m
axlen = max_len)
```

c. 음절 단위 토큰화

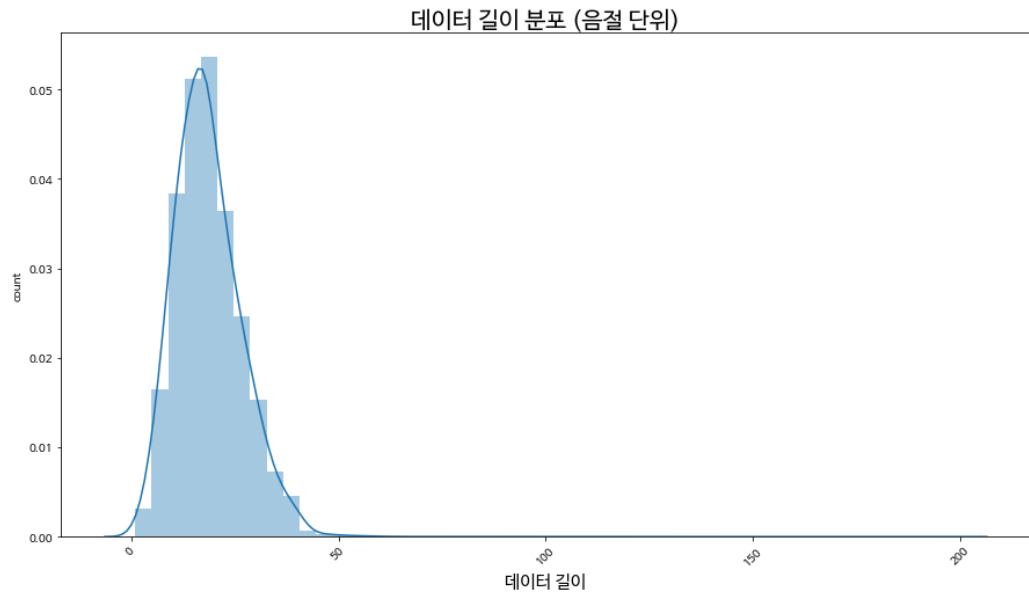
- i. 음절 단위로 문장을 끊어주는 syllable_tokenizer 함수를 만들어 토큰화를 진행했다.

```
def syllable_tokenizer(s):
    temp = s.replace(' ', '')
    result = []
    for c in temp:
        result.append(c)
    return result

X_train = []
for sentence in tqdm(train['records']):
    tokenized_sentence = syllable_tokenizer(sentence) # 토큰화
    X_train.append(tokenized_sentence)

X_test = []
for sentence in tqdm(test['records']):
    tokenized_sentence = syllable_tokenizer(sentence) # 토큰화
    X_test.append(tokenized_sentence)
```

- ii. 데이터 길이의 분포를 히스토그램으로 시각화한 결과 대부분의 데이터가 음절 50개 이하였다.



iii. 정수 인코딩

- 단어 집합(vocabulary)의 크기 : 2,061
- 등장 빈도가 2번 이하인 희귀 단어의 수: 459
- 단어 집합에서 희귀 단어의 비율: 22.3%
- 전체 등장 빈도에서 희귀 단어 등장 빈도 비율: 0.0997%
- 희귀 단어의 등장 빈도 비율이 낮아 단어 집합에서 등장 빈도 수 2 이하인 단어는 제거하고 인코딩을 진행하였다.

```
vocab_size = total_cnt
tokenizer = Tokenizer(vocab_size) # 단어 집합의 크기를 지정하여 빈도수 2 이하인 단어는 제거
X_train = tokenizer.texts_to_sequences(X_train) # 텍스트 시퀀스를 정수 시퀀스로 변환
X_test = tokenizer.texts_to_sequences(X_test)
```

iv. 패딩

전체 샘플 중 길이가 40 이하인 샘플의 비율이 99.8%로 데이터의 대부분을 차지했다. 따라서 패딩 길이를 40으로 설정했다.

```
max_len = 40
X_train = pad_sequences(X_train, maxlen = max_len)
X_test = pad_sequences(X_test, maxlen = max_len)
```

v. 프로토타입 모델

```
model = Sequential()
model.add(Embedding(vocab_size, 100))
model.add(LSTM(128))
model.add(Dense(1, activation='sigmoid'))

es = EarlyStopping(patience=4, restore_best_weights=True)
mc = ModelCheckpoint('./model/york.h5', save_best_only=True)

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=100,
                    callbacks=[es, mc], batch_size=64, validation_split=0.2)

york_model = load_model('./model/york.h5')
print("테스트 정확도: {:.4f}".format(york_model.evaluate(X_test, y_test)[1]))
# Accuracy: 0.9191
```

음절 단위 토큰화, LSTM 모델 코드

음절 단위로 토큰화하여 LSTM 신경망을 이용한 모델은 정확도 91.91%를 가졌지만, 테스트 결과 좋지 않은 성능을 보였다. 따라서 추후 모델링에 음절단위 토큰화는 사용하지 않기로 결정했다.

d. 자모 단위 토큰화

- i. 문장을 입력하면 자모단위로 분해하는 to_jaso() 함수를 만들어 자모단위 토큰화를 진행했다. 예를 들어 '피자 먹고 싶다' 라는 문장을 to_jaso 함수에 집어 넣으면 'ㅍㅣㅅㅈㅌㅁㅍㅌㅊㅊㅌㅈㅌㅣㅍㅌㅌㅌㅌ'를 반환한다. ㅅ는 종성이 비어있는 경우 넣어주는 기호이다.

```
NO_JONGSUNG = '8'

CHOSUNGS = ['ㄱ', 'ㄴ', 'ㄷ', 'ㄹ', 'ㅁ', 'ㅂ', 'ㅅ', 'ㅇ', 'ㅈ', 'ㅊ', 'ㅋ', 'ㆁ', 'ㅅ', 'ㅇ', 'ㅈ', 'ㅊ', 'ㅋ', 'ㆁ']
```



```

'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ', 'ㅁ', 'ㅂ', 'ㅅ', 'ㅇ', 'ㅈ', 'ㅊ', 'ㅋ', 'ㆁ',
'ㄷ', 'ㅌ', 'ㅍ', 'ㅎ']
JOONGSUNGS = ['ㅏ', 'ㅑ', 'ㅓ', 'ㅕ', 'ㅗ', 'ㅛ', 'ㅜ', 'ㅠ', 'ㅡ', 'ㅣ',
'ㅓ', 'ㅕ', 'ㅗ', 'ㅛ', 'ㅜ', 'ㅠ', 'ㅡ', 'ㅣ', 'ㅣ']
JONGSUNGS = [NO_JONGSUNG, 'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ',
'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ', 'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ', 'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ',
'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ', 'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ', 'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ',
'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ', 'ㄱ', 'ㄴ', 'ㄷ', 'ㄹ']

```

```

N_CHOSUNGS = 19
N_JOONGSUNGS = 21
N_JONGSUNGS = 28

```

```

FIRST_HANGUL = 0xAC00 #'가'
LAST_HANGUL = 0xD7A3 #'힣'

```

```

def to_jamo(s):
    result = []
    for c in s:
        if ord(c) < FIRST_HANGUL or ord(c) > LAST_HANGUL: # if a character is a hangul
            result.append(c)
        else:
            code = ord(c) - FIRST_HANGUL
            jongsung_index = code % N_JONGSUNGS
            code //= N_JONGSUNGS
            joongsung_index = code % N_JOONGSUNGS
            code //= N_JOONGSUNGS
            chosung_index = code

            result.append(CHOSUNGS[chosung_index])
            result.append(JOONGSUNGS[joongsung_index])
            result.append(JONGSUNGS[jongsung_index])

    return ''.join(result)

```

to_jamo 함수로 분해한 문장을 bunhae 함수에 입력해 자모단위, 1글자 단위로 분해한다. 자모 토큰화를 3가지 방식으로 진행했는데, 예를 들어 '피자 먹고 싶다'를 토큰화할 때 1. 'ㅍ|ㅅ|', 'ㅣ|ㅈ|ㅅ|', 'ㅁ|ㅊ|', 'ㄱ|ㅊ|ㅅ|', 'ㅣ|ㅅ|', 'ㅍ|ㅊ|ㅅ|'로 분해되는 방식, 2. 'ㅍ|ㅅ|ㅈ|ㅅ|ㅁ|ㅊ|ㅊ|ㅅ|ㅅ|ㅍ|ㅊ|ㅅ|'로 분해되는 방식, 3. 'ㅍ|', 'ㅣ|', 'ㅅ|', 'ㅣ|', 'ㅈ|', 'ㅣ|', 'ㅅ|', 'ㅁ|', 'ㅊ|', 'ㅣ|', 'ㅊ|', 'ㅅ|', 'ㅅ|', 'ㅍ|', 'ㅣ|', 'ㅍ|', 'ㅊ|', 'ㅣ|', 'ㅅ|'로 분해되는 방식이다. 본 프로젝트에서는 3번째 방식으로 토큰화하였다.

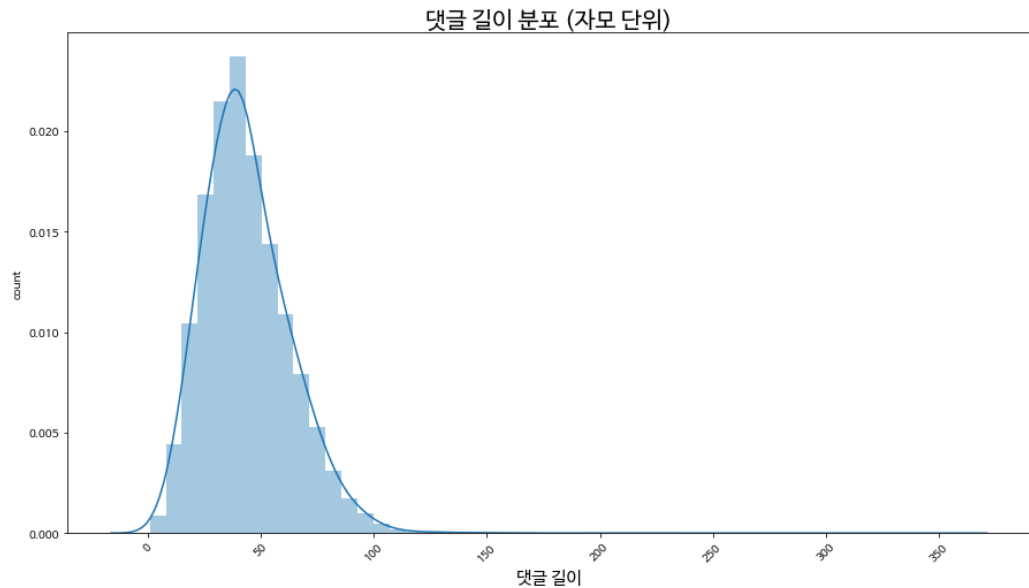
```
def bunhae(s):
    return [to_jamo(i) for i in s]

result1=[]
for i in range(len(df)):
    result1.append(bunhae(df.iloc[i,0]))
print(result1[:2])

result2=[]
for i in range(len(df)):
    result2.append(bunhae(["".join(df.iloc[i,
0]))))
print(result2[:2])

result3=[]
for i in range(len(df)):
    result3.append(list(result2[i][0]))
print(result3[:2])
```

- ii. 데이터의 길이를 히스토그램으로 시각화한 결과, 데이터의 99.4%가 자모 100개 이하였다.



iii. 정수 인코딩

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(result3)

word2idx = tokenizer.word_index
idx2word = {value : key for key, value in word2idx.items()}
encoded = tokenizer.texts_to_sequences(result3)
```

iv. 패딩

전체 샘플 중 길이가 100 이하인 샘플의 비율이 99.4%로 데이터의 대부분을 차지했다. 따라서 패딩 길이를 100으로 설정하고 진행했다.

```
max_len = 100
below_threshold_len(max_len, encoded)

encoded = pad_sequences(encoded, maxlen = max_len)

X_train, X_valid, y_train, y_valid = train_test_split(
    encoded, df['label'], test_size=0.2, random_state=123)
```

모델링

위에서 생성한 토큰화 데이터 중 프로토타입에서 성능이 좋았던 형태소 단위 토큰화 데이터, 자모 단위 토큰화 데이터를 이용한다. 게임 채팅의 경우 빠르게 전송해야 하기 때문에 오타가 많고, 욕설의 경우 필터링을 피하기 위해 숫자나 특수문자를 활용한 변형이 존재한다. 형태소 단위 토큰으로는 이와 같은 오타, 변형을 학습시키는 데 부족함이 있을 것으로 예상하여 자모 단위 토큰을 더 집중적으로 사용하였다.

임베딩 방식으로는 tensorflow의 Embedding 라이브러리와 FastText 라이브러리를 사용한다. FastText는 단어에 대한 벡터 표현을 생성해주는 라이브러리로, 학습된 내부 단어들을 통해 모르는 단어와의 유사도를 계산하여 비슷한 단어를 찾아낸다. 또한 n_gram이 겹치는 단어와 유사도를 계산해 임베딩 성능을 도출할 수 있어 노이즈가 많은 corpus에 강점을 가지고 있으며, 비교적 빈도수가 적은 오타 등의 희귀단어를 임베딩 할 때도 일정 수준의 성능을 보인다.

합성곱층으로는 LSTM, CNN을 이용한다. LSTM은 금칙어 리스트로 판별할 수 없는 맥락상의 욕설을 구분할 수 있고, CNN은 '시1발'고 같이 욕설 사이에 다른 문자를 넣어서 변형한 욕설을 판별할 수 있다.

모델 성능 테스트에는 데이터 베이스에 충분히 많은 욕설을 다양하게 변형하여 사용하였다.

1. 형태소 단위 토큰, LSTM

a. 모델 파라미터

- Dropout 비율 : 60%
- LSTM의 output 차원 : 100
- 출력층 활성화 함수 : sigmoid
- 최적화 함수 : adam
- 손실 함수 : binary_crossentropy
- 평가 지표 : accuracy
- 정확도 : 93.22%

```
model = Sequential()
model.add(Embedding(total_cnt, 100))
model.add(LSTM(100, dropout=0.6))
model.add(Dense(1, activation='sigmoid'))

es = EarlyStopping(patience=4, restore_best_weights=True)
mc_m = ModelCheckpoint('./model/best_model_mean_words_morpheme.h5', save_best_only=True)
```

```

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(morpheme_X_train, y_train, epochs=100, callbacks=[es, mc_m], batch_size=16, validation_split=0.2)

print("\n Accuracy: %.4f" % (model.evaluate(morpheme_X_test, y_test)[1]))

loaded_model_morpheme = load_model('./model/best_model_mean_words_morpheme.h5')

```

b. 욕설 판별

i. 단어 사전에 있는 욕설 : 잡아냄

```

mean_word_predict_morpheme('시발')
mean_word_predict_morpheme('씨발')

```

99.88% 확률로 욕설입니다.

99.83% 확률로 욕설입니다.

ii. 단어 사전에 없는 욕설 : 잡아내지 못함

```

mean_word_predict_morpheme('시2발')
mean_word_predict_morpheme('씨2발')

```

71.46% 확률로 욕설이 아닙니다.

54.34% 확률로 욕설이 아닙니다.

iii. 욕설 사이에 특수문자를 넣은 경우 : 잡아냄

```

mean_word_predict_morpheme('시@발')

```

91.28% 확률로 욕설입니다.

iv. 오타 : 잡아내지 못함

```
mean_word_predict_morpheme('ㅅㅅㅅㅅㅅ')  
mean_word_predict_morpheme('ㅅㅅㅅㅅ')  
mean_word_predict_morpheme('ㅅㅅㅅㅅㅅㅅ')
```

78.44% 확률로 욕설이 아닙니다.

62.06% 확률로 욕설이 아닙니다.

62.06% 확률로 욕설이 아닙니다.

v. 한계점

: '시@발'처럼 욕설 사이에 특수문자나 숫자를 넣어 변형한 경우, 욕설로 판별은 되지만 '시', '@', '발'로 욕설이 나뉘어 학습된다. 또한 오타가 난 욕설은 잡지 못한다.

2. 자모 단위 토큰, LSTM

a. 모델 파라미터

- Dropout 비율 : 60%
- LSTM의 output 차원 : 100
- 출력층 활성화 함수 : sigmoid
- 최적화 함수 : adam
- 손실 함수 : binary_crossentropy
- 평가 지표 : accuracy
- 정확도 : 93.22%

```
model = Sequential()  
model.add(Embedding(vocab_size, 100))  
model.add(LSTM(100, dropout=0.6))  
model.add(Dense(1, activation='sigmoid'))  
  
es = EarlyStopping(patience=4, restore_best_weights=True)  
mc = ModelCheckpoint('./model/test1.h5', save_best_only=True)
```

```

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=100, callbacks=[es, mc], batch_size=128, validation_split=0.2)

from tensorflow.keras.models import load_model
loaded_model = load_model('./model/test1.h5')

print("테스트 정확도: {:.4f}".format(loaded_model.evaluate(X_valid, y_valid)[1]))

```

b. 욕설 판별

i. 단어 사전에 있는 욕설 : 잡아냄

```

sentiment_predict('시발')
sentiment_predict('씨발')
sentiment_predict('시1발')

```

99.56% 확률로 욕설입니다.

99.75% 확률로 욕설입니다.

98.33% 확률로 욕설입니다.

ii. 단어 사전에 없는 욕설 : 잡아냄

```

sentiment_predict('시2발')
sentiment_predict('씨2발')

```

99.27% 확률로 욕설입니다.

99.77% 확률로 욕설입니다.

iii. 욕설 사이에 특수문자를 넣은 경우 : 잡아냄

```
sentiment_predict('시@발')  
sentiment_predict('시!발')
```

99.43% 확률로 욕설입니다.

81.01% 확률로 욕설입니다.

iv. 오타 : 잡아내지 못함

```
sentiment_predict('ㅅㅅㅅㅅㅅㅅ')  
sentiment_predict('ㅅㅅㅅㅅㅅㅅㅅㅅ')
```

94.15% 확률로 욕설이 아닙니다.

59.79% 확률로 욕설이 아닙니다.

v. 한계점

: '시@발'처럼 욕설 사이에 특수문자나 숫자를 넣어 변형한 경우, 욕설로 판별은 되지만 '시', '@', '발'로 욕설이 나뉘어 학습된다. 또한 오타가 난 욕설은 잡지 못한다.

3. 자모 단위 토큰, FastText, LSTM

a. FastText

- corpus가 뭔지
- fasttext에 어떤 옵션 썼는지

```
corpus_fname = '/content/edittext.txt'  
model_fname = '/content/final.model'  
  
logging.basicConfig(format='%(asctime)s : %(levelname)s : %(message)s', level=logging.INFO)  
  
print('corpus 생성')  
corpus = [sent.strip().split(" ") for sent in tqdm(open(corpus_fname, 'r', encoding='utf-8').readlines())]  
  
print("학습 중")  
model = FastText(corpus, vector_size=100, min_count=
```



```
1, window=3, sg=1, negative=20)
model.save(model_fname)
```

b. 모델 파라미터

- Dropout 비율 : 60%
- LSTM의 output 차원 : 100
- 출력층 활성화 함수 : sigmoid
- 최적화 함수 : adam
- 손실 함수 : binary_crossentropy
- 평가 지표 : accuracy
- 정확도 : 93.22%

```
model = Sequential()
model.add(Embedding(vocab_size, 100))
model.add(LSTM(100, dropout=0.6))
model.add(Dense(1, activation='sigmoid'))

es = EarlyStopping(patience=4, restore_best_weights=True)
mc = ModelCheckpoint('./model/test1.h5', save_best_only=True)

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=100, callbacks=[es, mc], batch_size=128, validation_split=0.2)

from tensorflow.keras.models import load_model
loaded_model = load_model('./model/test1.h5')

print("테스트 정확도: {:.4f}".format(loaded_model.evaluate(X_valid, y_valid)[1]))
```

c. 욕설 판별

i. 단어 사전에 있는 욕설 : 잡아냄

```
mean_word_predict_morpheme("시발")
mean_word_predict_morpheme("씨발")
```

100%|██████████| 1/1 [00:00<00:00, 5289.16it/s]
100%|██████████| 1/1 [00:00<00:00, 10485.76it/s]
98.82% 확률로 욕설입니다.

100%|██████████| 1/1 [00:00<00:00, 2763.05it/s]
100%|██████████| 1/1 [00:00<00:00, 8559.80it/s]
99.01% 확률로 욕설입니다.

ii. 단어 사전에 없는 욕설 : 잡아냄

```
mean_word_predict_morpheme("시2발")
mean_word_predict_morpheme("씨2발")
```

100%|██████████| 1/1 [00:00<00:00, 3634.58it/s]
100%|██████████| 1/1 [00:00<00:00, 10894.30it/s]
88.26% 확률로 욕설입니다.

100%|██████████| 1/1 [00:00<00:00, 4306.27it/s]
100%|██████████| 1/1 [00:00<00:00, 9362.29it/s]
92.03% 확률로 욕설입니다.

iii. 욕설 사이에 특수문자를 넣은 경우 : 잡아냄

```
mean_word_predict_morpheme("시@발")
mean_word_predict_morpheme("시!발")
```

100%|██████████| 1/1 [00:00<00:00, 4660.34it/s]
100%|██████████| 1/1 [00:00<00:00, 9157.87it/s]
88.59% 확률로 욕설입니다.

100%|██████████| 1/1 [00:00<00:00, 5096.36it/s]
100%|██████████| 1/1 [00:00<00:00, 135.57it/s]88.41% 확률로 욕설입니다.

iv. 오타 : 잡아냄

```
mean_word_predict_morpheme("ㅅ ㅈ ㅊ ㅌ ")
mean_word_predict_morpheme("ㅊ ㅈ ㅅ ㅈ ㅊ ㅌ ")
```

```
100%|██████████| 1/1 [00:00<00:00, 5011.12it/s]
100%|██████████| 1/1 [00:00<00:00, 10538.45it/s]
78.97% 확률로 욕설입니다.
```

```
100%|██████████| 1/1 [00:00<00:00, 4032.98it/s]
100%|██████████| 1/1 [00:00<00:00, 10837.99it/s]64.34% 확률로 욕설입니다.
```

v. 한계점

: 단어간의 유사성을 기준으로 욕설 여부를 판단하기 때문에, 기존에 존재하던 데이터와 연관성이 떨어지는 새로운 욕의 경우 욕설로 판별하지 못한다.

4. 자모 단위 토큰, CNN

a. 모델 파라미터

- Dropout 비율 : 50%
- LSTM의 output 차원 : 216
- 출력층 활성화 함수 : sigmoid
- 최적화 함수 : adam
- 손실 함수 : binary_crossentropy
- 평가 지표 : accuracy
- 정확도 : 93.7%

```
embedding_dim = 64
dropout_ratio = 0.5
num_filters = 500
kernel_size = 10

model = Sequential()
model.add(Embedding(vocab_size, embedding_dim))
model.add(Dropout(dropout_ratio))
model.add(Conv1D(num_filters, kernel_size, padding='same', activation='relu'))
model.add(GlobalMaxPooling1D())
model.add(Dropout(dropout_ratio))
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

```

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['acc'])
es = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=3)
mc = ModelCheckpoint('best_model.h5', monitor='val_acc', mode='max', verbose=1, save_best_only=True)

history = model.fit(X_train, y_train, epochs=100, batch_size=128, validation_split=0.2, callbacks=[es, mc])
from tensorflow.keras.models import load_model
loaded_model = load_model('/content/best_model.h5')
print("테스트 정확도: {:.4f}".format(loaded_model.evaluate(X_valid, y_valid)[1]))

```

b. 욕설 판별

i. 단어 사전에 있는 욕설

```

sentiment_predict('시발')
sentiment_predict('씨발')

```

99.88% 확률로 욕설입니다.

99.94% 확률로 욕설입니다.

ii. 단어 사전에 없는 욕설

```

sentiment_predict('시2발')
sentiment_predict('씨2발')

```

52.85% 확률로 욕설이 아닙니다.

82.39% 확률로 욕설입니다.

iii. 욕설 사이에 특수문자를 넣은 경우

```
sentiment_predict('시@발')
sentiment_predict('시!발')
```

92.54% 확률로 욕설입니다.

50.66% 확률로 욕설이 아닙니다.

iv. 오타

```
sentiment_predict('ㅅㅅㅅㅅㅅㅅ')
sentiment_predict('ㅅㅅㅅㅅㅅㅅㅅㅅ')
```

68.97% 확률로 욕설입니다.

69.19% 확률로 욕설입니다.

v. 한계점

: 기존 모델에서 잡지 못했던 욕을 식별해냈으나, 욕설 중 데이터가 적은 경우는 자모 LSTM이 더 판별기능이 좋았다.

보팅

패스트텍스트를 이용한 LSTM 모델은 점수가 낮지만, 학습된 욕들에 대해서는 판별력이 좋았다. CNN 모델은 유사한 형태의 욕설을 잡아내지 못하는 문제가 생겼다. 각 모델마다 장단점이 있기 때문에 소프트보팅으로 서로의 판단 결과를 보완시키기로 했다. 1차적으로 자모 단위 패스트텍스트 LSTM이 욕설을 판별하도록 하고, 패스트텍스트 LSTM 모델이 욕설이라고 판별하면 욕설로 처리했다. 욕설이 아니라고 판단하면 자모 단위 LSTM, 자모 단위 CNN으로 소프트보팅을 진행해 욕설 여부를 다시 판별한다. 이때 소프트보팅이 욕설로 판단하면 욕설로 처리한다. 형태소 단위 LSTM은 특수문자로 변형한 '시@발'의 경우 '시', '@', '발'로 쪼개서 학습시키기 때문에 욕설이 아닌 단어도 욕설로 판단하는 등 모델 성능을 저하시킬 우려가 있어 보팅에서 제거했다.

```
new_sentence1 = okt.morphs(new_sentence, stem=True) # 토큰화
encoded1 = tokenizer1.texts_to_sequences([new_sentence1]) #
정수 인코딩
pad_new1 = pad_sequences(encoded1, maxlen = 10) # 패딩
score1 = float(loader_model1.predict(pad_new1)) # 예측

new_sentence2 = list("".join(bunhae(new_sentence)))
```

```

encoded2 = tokenizer2.texts_to_sequences([new_sentence2]) #
정수 인코딩
pad_new2 = pad_sequences(encoded2, maxlen = 100) # 패딩
score2 = float(loader_model2.predict(pad_new2)) # 예측

new_sentence3 = list("".join(bunhae(new_sentence)))
encoded3 = tokenizer3.texts_to_sequences([new_sentence3]) #
정수 인코딩
pad_new3 = pad_sequences(encoded3, maxlen = 100) # 패딩
score3 = float(loader_model3.predict(pad_new3)) # 예측

new_sentence4 = [new_sentence]
pd.DataFrame(new_sentence4).to_csv('jamofastlstm/sentence.t
xt', index = False, header = False)
sentence_txt = 'jamofastlstm/sentence.txt'
new_sentence_txt = 'jamofastlstm/newsentence.txt'
process_jamo(sentence_txt, new_sentence_txt)
new_sentence_final = [sent.strip().split(" ") for sent in t
qdm(open(new_sentence_txt, 'r', encoding='utf-8').readlines
())]
new_sentencelist = []
for a in new_sentence_final:
    new_sentencelist2 = []
    for word in a:
        ft_modeled = model_morphs.wv.get_vector(word)
        new_sentencelist2.append(ft_modeled)
    new_sentencelist.append(new_sentencelist2)
score4 = float(loader_model4.predict(pad_sequences(new_sent
encelist, maxlen = 15, padding='post', truncating = 'post',
dtype = float )))

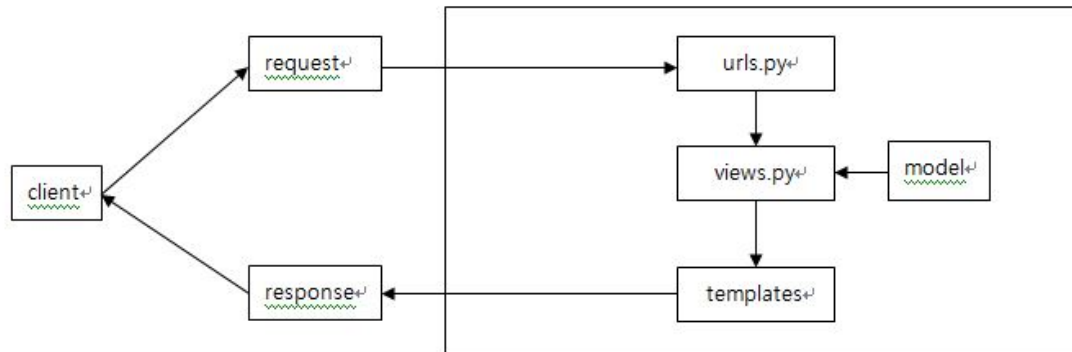
s_mean = ( score2 + score3 ) / 2
if score4 >= 0.5:
    return ('욕설이 감지되었습니다.')
else:
    if s_mean > 0.5:
        return ('욕설이 감지되었습니다.')

```

```
else:
    return ('욕설이 감지되지 않았습니다.')
```

2-4. 웹페이지 구축(Django)

구조



단계

1. yokprj의 urls.py에 yok이라는 app 이름을 추가

```
python manage.py startapp yok
```

2. yok의 urls.py : 페이지 요청이 발생하면 가장 먼저 호출되는 파일로 URL과 뷰 함수 (views.py 파일에 정의된 함수) 간의 매핑을 정의

```
urlpatterns = [
    path('', views.index, name='index'),
    path('predict', views.predict, name="predict"), # predict URL 요청되면 views.py에 있는 predict를 호출
]
```

- index : 문장을 입력하고 확인 버튼을 누르는 함수
- predict : 문장을 모델에 입력해서 결과값을 리턴하는 함수

3. views.py에서 index.html를 불러와서 화면에 뿌려주도록 하기 : index.html는 문장을 입력하고 확인 버튼을 누르는 html

```
def index(request):
    return render(request, 'index.html')
```

4. views.py에서 h5 모델과 단어사전을 불러오기(load_model)

- h5 모델은 load_model로 불러오기
- 단어사전들은 pickle의 load로 불러오기

```
file1 = os.path.dirname(__file__) + '\\wordlstm.h5'
file2 = os.path.dirname(__file__) + '\\jamolstm.h5'
file3 = os.path.dirname(__file__) + '\\jamocnn.h5'

loaded_model1 = load_model(file1) # LSTM 형태소 모델 로드
loaded_model2 = load_model(file2) # LSTM 자소 모델 로드
loaded_model3 = load_model(file3) # CNN 자소 모델 로드

pickle1 = os.path.dirname(__file__) + '\\wordlstm.pkl'
pickle2 = os.path.dirname(__file__) + '\\jamolstm.pkl'
pickle3 = os.path.dirname(__file__) + '\\jamocnn.pkl'

with open(pickle1, 'rb') as f: # LSTM 형태소 단어사전 로드
    tokenizer1 = pickle.load(f)

with open(pickle2, 'rb') as f: # LSTM 자모 단어사전 로드
    tokenizer2 = pickle.load(f)

with open(pickle3, 'rb') as f: # CNN 자모 단어사전 로드
    tokenizer3 = pickle.load(f)
```

5. input 값이 들어오면 토큰화, 정수인코딩, 패딩한 다음, 불러온 모델로 예측한다. 총 3개의 모델을 가지고 각각 예측한 다음, 각 예측한 값들의 평균을 내서 욕인지 아닌지를 판별하는 predict_test 함수 만들기

```
new_sentence1 = okt.morphs(new_sentence, stem=True) # 토큰화
encoded1 = tokenizer1.texts_to_sequences([new_sentence1])
# 정수 인코딩
```



```
pad_new1 = pad_sequences(encoded1, maxlen = 10) # 패딩
score1 = float(loader_model1.predict(pad_new1)) # 예측
```

6. 3개의 모델의 예측값들을 평균으로 계산한 다음, 평균값과 결과내용을 result.html에 넘겨주기

```
score_average=(score1+score2+score3)/3
if(score_average > 0.5):
    return (" {:.2f}% 확률로 욕설입니다.\n".format(score_average * 100))
else:
    return (" {:.2f}% 확률로 욕설이 아닙니다.\n".format((1 - score_average) * 100))
```

```
return render(request, 'result.html', {'Name': name1,})
```

7. 넘겨준 값과 내용들을 웹페이지에 보여지도록 하기

```
<body>
    <h1>결과: {{result}}</h1>

    # 웹 페이지에 표시할 이미지
    
    

    <button onclick="myFunction()">확인</button>

    <p id="demo"></p>

    <script>
    function myFunction() {
        let text = "맞게 예측했습니까?";
        if (confirm(text) == true) {
            text = "1"; # 욕일 시 1
        } else {
            text = "0"; # 욕이 아닐 시 0
        }
        document.getElementById("demo").innerHTML = text;
```

```

# 'demo'라는 Id 요소를 찾아서 text 변수를 웹페이지에 출력하기.
}
</script>

</body>

```

서버 화면

1. 욕설 입력창 : 이름, 욕설 입력창, 작성 버튼 생성

작성자	입력한 문장	판별결과
o o	수박을먹을때는씨발라먹어	욕설입니다.
o o	유아연씨밥세끼챙겨드세요	욕설입니다.
o o	비나안개같은날씨발밑조심	욕설입니다.
o o	플로리다마이애미국제공항	욕설입니다.

2. 욕설 입력 결과 : 입력한 이름, 욕설이 서버 화면에 추가되고 판별결과를 욕설입니다 / 욕설이 아닙니다로 나타냄

a. 욕설을 입력한 경우

작성자	입력한 문장	판별결과
o o	수박을먹을때는씨발라먹어	욕설입니다.
o o	유아인씨밥세끼챙겨드세요	욕설입니다.
o o	비나안개같은날씨발밑조심	욕설입니다.
o o	플로리다마이애미국제공항	욕설입니다.
테스트	씨~발람아	욕설입니다.

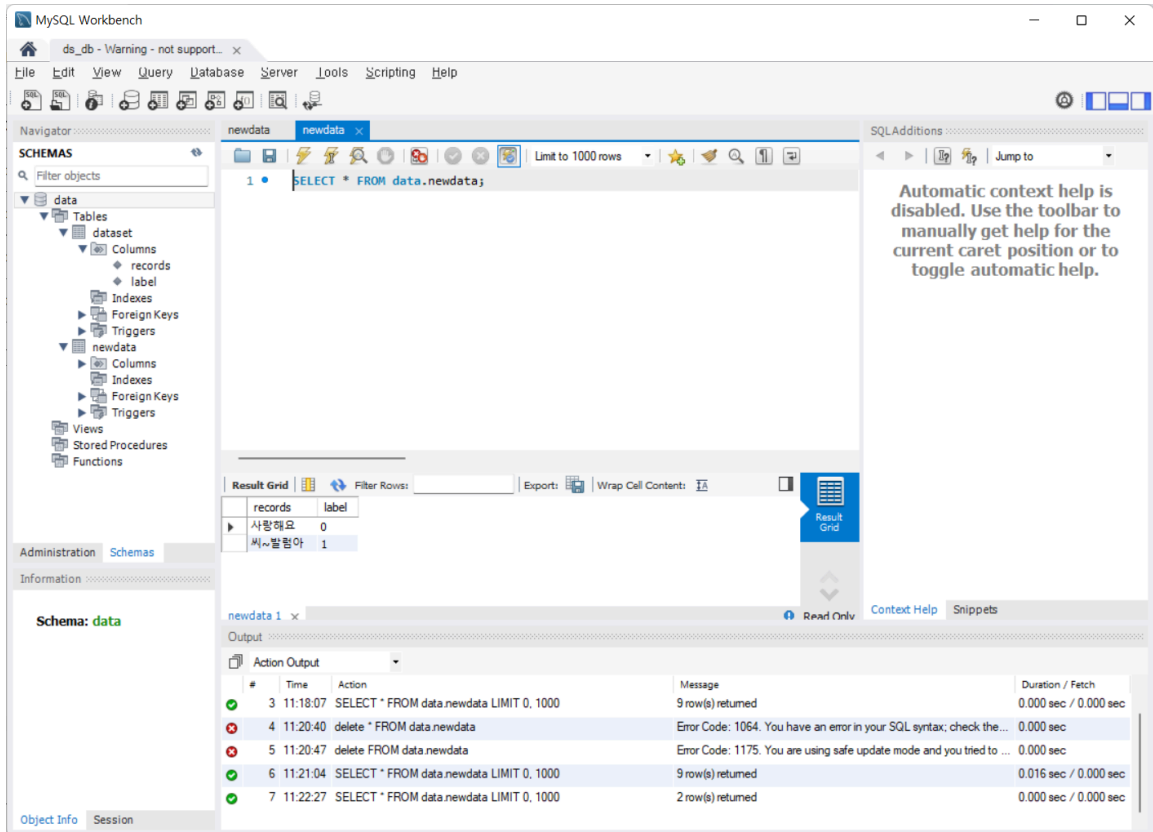
이름 욕설 입력

b. 비욕설을 입력한 경우

작성자	입력한 문장	판별결과
o o	수박을먹을때는씨발라먹어	욕설입니다.
o o	유아인씨밥세끼챙겨드세요	욕설입니다.
o o	비나안개같은날씨발밑조심	욕설입니다.
o o	플로리다마이애미국제공항	욕설입니다.
테스트	사랑해요	욕설이 아닙니다.

이름 욕설 입력

3. MySQL 데이터베이스 : 입력한 욕설과 욕설 판별결과(욕설이면 1, 욕설이 아니면 0)가 서버와 연결된 데이터베이스에 자동으로 입력



03. 향후 개선 및 기대효과

‘또라이’ 등 커뮤니티에서 잘 쓰이지 않는 욕설은 모델이 욕설로 감지하지 못하는 경우가 있었고, 욕설이 아닌 단어가 욕설과 같은 문장에 쓰인 빈도가 높아 욕설로 감지되는 경우가 있었다. 이는 데이터의 수가 부족해서 학습이 모자라 일어난 현상으로, 탐지하지 못한 욕을 데이터베이스에 추가해 주기적으로 모델을 학습시키는 방안으로 개선할 수 있을 것이다.

데이터 수집처의 경우, 게임 채팅 데이터를 수집하기 어려워 온라인 커뮤니티로 한정했다. 게임과 커뮤니티 글쓰기의 특성은 다르므로 게임 채팅의 특성을 모델에 학습시키지 못했다. 게임 채팅 데이터를 추가로 학습시킨다면 성능이 더욱 좋아질 것이다.

데이터 라벨링의 경우, 사람이 일일이 하는 것이기 때문에 같은 기준으로 진행한다고 해도 담당자에 따라 라벨링 결과가 달라지는 경우가 있었다. 교차검증을 하거나, 비속어가 아닌 맥락상의 비방이면 한 사람이 라벨링을 진행한다면 정확도가 올라갈 것이다.

특수기호나 타국 언어를 조합하여 기존에 존재하던 욕설과 유사한 형태로 변형한 욕의 경우, 데이터 전처리를 위한 딥러닝 모델을 추가해 욕설 판별 모델과 함께 사용한다면 성능이 더 좋아질 것이다. 예를 들어, ‘^^ | 발’이라는 욕을 텍스트에서 이미지로 바꾸고, 그 이미지에서 ‘씨발’이라는 욕을 인식하여 다시 텍스트로 바꾸는 식이다. 이 경우 판별 모델은 변형되지 않은 단순한 형태의 욕설만 판별하면 되기 때문에 더 간단한 알고리즘으로 더 좋은 정확도를

가질 수 있을 것이다.

또한 본 프로젝트에서는 모델 하나당 합성곱 하나만 사용했으나, 모델 하나에 합성곱 여러 개를 쓴다면 다수의 특징을 한 번에 잡아낼 수 있을 것이다. 예를 들어 CNN과 LSTM을 동시에 사용한다면, 특수문자를 이용해 변형한 욕과 문맥상의 욕을 모델 하나만 사용하여 모두 탐지하여 인터넷 문화 개선에 기여할 수 있을 것으로 기대된다.

04. 개발 후기 및 느낀점

- 김주영 : 데이터 수집할 때, 수작업으로 라벨링한다고 해서 쉬울 줄 알았는데, 육체적인 노동이 많이 필요해서 힘들었다. 자연어 처리에 관심이 있어서 모델을 한 번 직접 시도해보려고 했지만 생각보다 접근하기가 어려웠다. 그래서 시간이 많이 걸릴 거 같아 장고부터 시작했고 가끔 밤새가면서 개발했다. 어느 정도 결과값이 나오니까 뿌듯하기도 했다.
- 박미현 : 모델링 과정이 생각보다 지리해서 잘 하고 있는 건지 의문이 들었지만 팀원들의 도움을 받아 해결할 수 있었고, 자연어처리를 직접 해보면서 여러 라이브러리와 알고리즘을 공부해 프로젝트 전보다 아는 것이 많아진 것 같다.
- 백지현 : 다양한 작업이 요구되는 딥러닝 프로젝트에서 팀원간 역할분담과 협업을 통해 효율적으로 업무를 해결하며 협업의 중요성과 유용성을 체감하였다. 부족한 실력으로 좋은 팀원들을 만나서 많이 배웠다.
- 송현승 : 시작하기 전에는 세상에 모든 데이터가 있고 쉽게 분석할 수 있을 것으로 생각했지만 프로젝트를 진행하면서 내가 정말로 원하는 모형의 데이터는 세상에 존재하지 않는다는 것을 느꼈다. 좋은 데이터 하나 열 데이터 안 부럽다.
- 이창용 : 나 아닌 다른 팀원과 같이하기에 소통이 많이 불편했다. 그럼에도 불구하고 역할을 분담해서 하기에 내 파트에 온전히 집중할 수 있었던 부분은 정말 편했다. 마감일이 다가와야 머리가 잘 돌아간다.
- 제갈덕 : 기획 단계에서 시간 설정이 얼마나 중요한지를 새삼 깨닫게 해 준 프로젝트였습니다. 다음엔 기획 단계에서 각각의 시간 설정을 좀 더 알아보고 진행해야 겠다고 생각했습니다. 데이터 수집 과정과 처리 과정의 노고를 300% 몸으로 느낄 수 있었고, 스스로의 부족함에 대해서 다시 되돌아보는 계기가 되었습니다.